Lecture6a

October 5, 2025

1 CS5489- Machine Learning

2 Lecture 6a - Convolutional Neural Networks (CNNs)

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

- Convolutional neural network (CNN)
- Regularization

[1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
from scipy import signal
from prettytable import PrettyTable

rbow = plt.get_cmap('rainbow')
```

[2]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import struct
import sys
print(f"Python: {sys.version} PyTorch: {torch.__version__}")

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Python: 3.13.7 | packaged by Anaconda, Inc. | (main, Sep  9 2025, 19:54:17)
[Clang 17.0.6 ] PyTorch: 2.8.0
Using device: cpu
```

[3]:
```
def plot_history(history):
    fig, ax1 = plt.subplots()

    ax1.plot(history['train_loss'], 'r', label="training loss ({:.6f})".
 ↪format(history['train_loss'][-1]))
    ax1.plot(history['val_loss'], 'r--', label="validation loss ({:.6f})".
 ↪format(history['val_loss'][-1]))
    ax1.grid(True)
    ax1.set_xlabel('iteration')
    ax1.legend(loc="best", fontsize=9)
    ax1.set_ylabel('loss', color='r')
    ax1.tick_params('y', colors='r')

    if 'train_accuracy' in history:
        ax2 = ax1.twinx()

        ax2.plot(history['train_accuracy'], 'b', label="training acc ({:.4f})".
 ↪format(history['train_accuracy'][-1]))
        ax2.plot(history['val_accuracy'], 'b--', label="validation acc ({:.
 ↪4f})".format(history['val_accuracy'][-1]))

        ax2.legend(loc="best", fontsize=9)
        ax2.set_ylabel('acc', color='b')
        ax2.tick_params('y', colors='b')
```

[4]:
```
def show_imgs(W_list, nc=10, highlight_green=None, highlight_red=None,␣
 ↪titles=None):
    nfilter = len(W_list)
    nr = (nfilter - 1) // nc + 1
    for i in range(nr):
        for j in range(nc):
            idx = i * nc + j
            if idx == nfilter:
                break
            plt.subplot(nr, nc, idx + 1)
            cur_W = W_list[idx]
            plt.imshow(cur_W,cmap='gray', interpolation='nearest')
            if titles is not None:
                plt.title(titles % idx)

            if ((highlight_green is not None) and highlight_green[idx]) or \
               ((highlight_red is not None) and highlight_red[idx]):
                ax = plt.gca()
```

```
                if highlight_green[idx]:
                    mycol = '#00FF00'
                else:
                    mycol = 'r'
                for S in ['bottom', 'top', 'right', 'left']:
                    ax.spines[S].set_color(mycol)
                    ax.spines[S].set_lw(2.0)
                ax.xaxis.set_ticks_position('none')
                ax.yaxis.set_ticks_position('none')
                ax.set_xticks([])
                ax.set_yticks([])
            else:
                plt.gca().set_axis_off()
```

[5]:
```
def read_32int(f):
    return struct.unpack('>I', f.read(4))[0]
def read_img(img_path):
    with open(img_path, 'rb') as f:
        magic_num = read_32int(f)
        num_image = read_32int(f)
        n_row = read_32int(f)
        n_col = read_32int(f)
        #print 'num_image = {}; n_row = {}; n_col = {}'.format(num_image,␣
 ↪n_row, n_col)
        res = []
        npixel = n_row * n_col
        res_arr = fromfile(f, dtype='B')
        res_arr = res_arr.reshape((num_image, n_row, n_col), order='C')
        #print 'image data shape = {}'.format(res_arr.shape)
        return num_image, n_row, n_col, res_arr
def read_label(label_path):
    with open(label_path, 'rb') as f:
        magic_num = read_32int(f)
        num_label = read_32int(f)
        #print 'num_label = {}'.format(num_label)
        res_arr = fromfile(f, dtype='B')
        #print res_arr.shape
        #res_arr = res_arr.reshape((num_label, 1))
        res_arr = res_arr.ravel()
        #print 'label data shape = {}'.format(res_arr.shape)
        return num_label, res_arr
```

[6]:
```
n_train, nrow, ncol, trainimg = read_img('data/train-images.idx3-ubyte')
_, trainY = read_label('data/train-labels.idx1-ubyte')
n_test, _, _, testimg = read_img('data/t10k-images.idx3-ubyte')
_, testY = read_label('data/t10k-labels.idx1-ubyte')
```

```
# for demonstration we only use 10% of the training data
sample_index = range(0, trainimg.shape[0], 10)
trainimg  = trainimg[sample_index]
trainY    = trainY[sample_index]
print(trainimg.shape)
print(trainY.shape)
print(testimg.shape)
print(testY.shape)
```

```
(6000, 28, 28)
(6000,)
(10000, 28, 28)
(10000,)
```

4 Signals

- So far we have assumed the input $\mathbf{x}$ is a vector
  - or have turned 2D images into vectors.
- *What if the input has more structure?*
- For example:
  - **1-D signal** (time)

mono audio (1 feature)

stereo audio (2 features)

<tr><td><img src="imgs/audio1.png" width=400></td><td><img src="imgs/audio2.jpg" width=400></td

- **2-D signal** (space)
  grayscale image (1 feature)
  color image (3 features)
  hyperspectral image (300 features)
- **3-D signal** (space+time, volume)

color video (3 features)

3D CT scan (1 feature)

<tr><td><img src="imgs/video.png" width=300></td><td><img src="imgs/ct.jpeg" width=300></td></t

5 Assumed Properties of Signals

- *Locality*
  - at low-level, features from 1 region are independent (do not depend on) features from a far-away region.
- *Translation*
  - the same features can appear anywhere in the signal.

6 Using the standard MLP layer...

- Each feature z_i is computed from all the inputs, but we only want local features (locality).
- The pattern could appear anywhere, but weights are trained for each location z_i separately (translation).
- For image input, we transform the image into a vector, which is the input into the MLP.
- **Problem:** This ignores the spatial relationship between pixels in the image.
  - Images contain local structures
    - groups of neighboring pixels correspond to visual structures (edges, corners, texture).
    - pixels far from each other are typically not correlated.
- **How to model the local structure of the signal?**
  - local features, feature translation
- **Answer**: Use a local feature extractor in the signal.

7 Convolution

- Consider 1D signal in *discrete* time: $x(t)$, $t \in \mathbb{Z}$
- Define the filter $w(t)$
  - "flipped" filter: $\tilde{w}(t) = w(-t)$

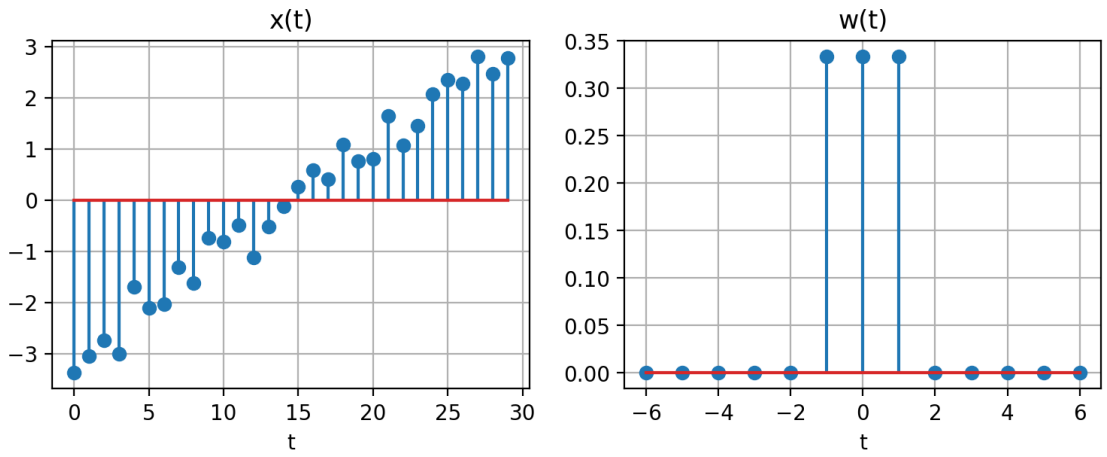
[7]:
```
def zeropad(w, left0, right0):
    return concatenate([zeros(left0,), w, zeros(right0,)])

def zeropad2(w, left0, right0, up0, down0):
    ws = w.shape;
    x = concatenate([
        zeros(shape=(up0,left0+ws[1]+right0)),
        concatenate([
            zeros(shape=(ws[0], left0)),
            w,
            zeros(shape=(ws[0], right0))
        ], axis=1),
        zeros(shape=(down0, left0+ws[1]+right0))
    ])
    return x
```

[8]:
```
random.seed(4487)
x = linspace(-3,3,30) + 0.3*random.normal(size=(30,))
w = [1/3, 1/3, 1/3]
ww = zeropad(w,5,5)
tw = flip(w)

xfig = plt.figure(figsize=(9,3))
plt.subplot(1,2,1)
plt.stem(x)
```

```
plt.grid(True)
plt.xlabel('t')
plt.title('x(t)')
plt.subplot(1,2,2)
plt.stem(arange(-6,7), ww)
plt.xlabel('t')
plt.title('w(t)')
plt.grid(True)
plt.close()
```

[9]:
```
xfig
```

[9]:



- Convolution is a filtering operation
  - $s(t) = x * w = \sum_a x(a)w(t-a)$
    - "$*$" is the symbol for convolution
- It's related to cross-correlation with the "flipped" filter.
  - $s(t) = \sum_a x(a)\tilde{w}(a-t)$
  - for a given t:
    1. shift $\tilde{w}$ by t
    2. multiply shifted $\tilde{w}$ with x
    3. sum to get $s(t)$

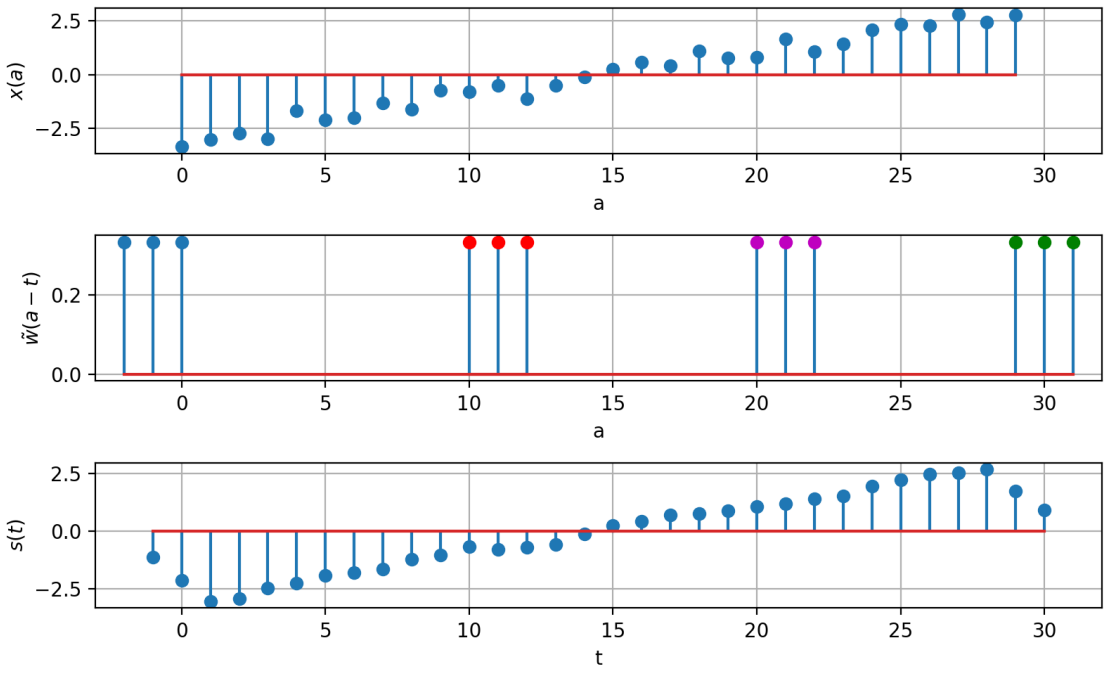
[10]:
```
s = convolve(x, w)

sfig = plt.figure(figsize=(8,5))
plt.subplot(3,1,1)
plt.stem(x)
plt.xlim(-3,32); plt.grid(True)
plt.ylabel('$x(a)$')
plt.xlabel('a')
plt.subplot(3,1,2)
```

```
plt.stem(range(-2,1), tw)
plt.stem(range(10,13), tw, markerfmt='or')
plt.stem(range(20,23), tw, markerfmt='om')
plt.stem(range(29,32), tw, markerfmt='og')
plt.xlim(-3,32); plt.grid(True)
plt.ylabel('$\\tilde{w}(a-t)$')
plt.xlabel('a')
plt.stem(zeros(30,), markerfmt=',r')
plt.subplot(3,1,3)
plt.stem(range(-1,31), s)
plt.xlim(-3,32); plt.grid(True)
plt.ylabel('$s(t)$')
plt.xlabel('t')
#plt.subplots_adjust(hspace=0.35)
plt.tight_layout()
plt.close()
```

- $s(t) = \sum_a x(a)\tilde{w}(a-t)$
  1. shift $\tilde{w}$ by t
  2. multiply shifted $\tilde{w}$ with x
  3. sum to get $s(t)$

[11]:
```
sfig
```

[11]:



- Boundary conditions

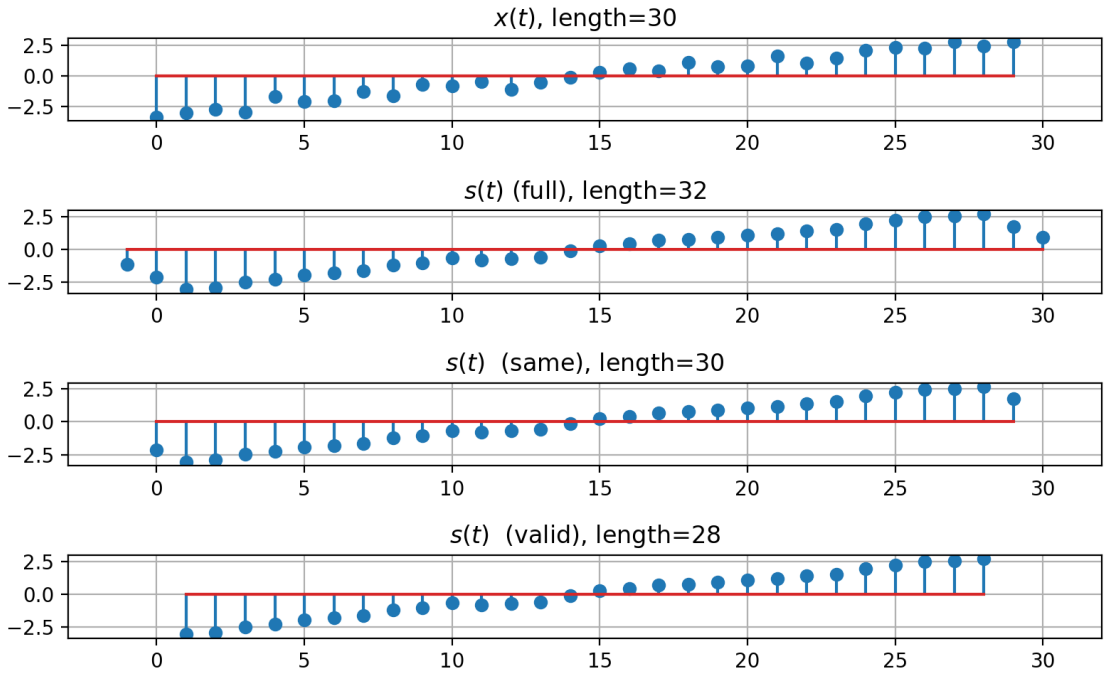
  - It is assumed that the rest of the signal is all 0.
- The convolution result near the ends of the signal uses these "artificial" zeros.
  - the filtered signal is longer than the original signal
  - length increases by $P-1$, where P is the non-zero extent of the filter.

- Three ways to handle this:
  1) use everything with *non-zero* response ('full' mode)
  2) keep the response the *same* length as the signal ('same' mode)
  3) keep only responses where the *entire* filter is on the signal ('valid' mode)

[12]:
```
cfig = plt.figure(figsize=(8,5))
plt.subplot(4,1,1)
plt.stem(x)
plt.xlim(-3,32); plt.grid(True)
plt.title('$x(t)$, length=30')
plt.subplot(4,1,2)
plt.stem(range(-1,31), convolve(x,w,mode='full'))
plt.xlim(-3,32); plt.grid(True)
plt.title('$s(t)$ (full), length=32')
plt.subplot(4,1,3)
plt.stem(range(0,30), convolve(x,w,mode='same'))
plt.xlim(-3,32); plt.grid(True)
plt.title('$s(t)$  (same), length=30')
plt.subplot(4,1,4)
plt.stem(range(1,29), convolve(x,w,mode='valid'))
plt.xlim(-3,32); plt.grid(True)
plt.title('$s(t)$  (valid), length=28')

#plt.subplots_adjust(hspace=0.50)
plt.tight_layout()
plt.close()
```

[13]:
```
cfig
```

[13]:



8 2D Convolution

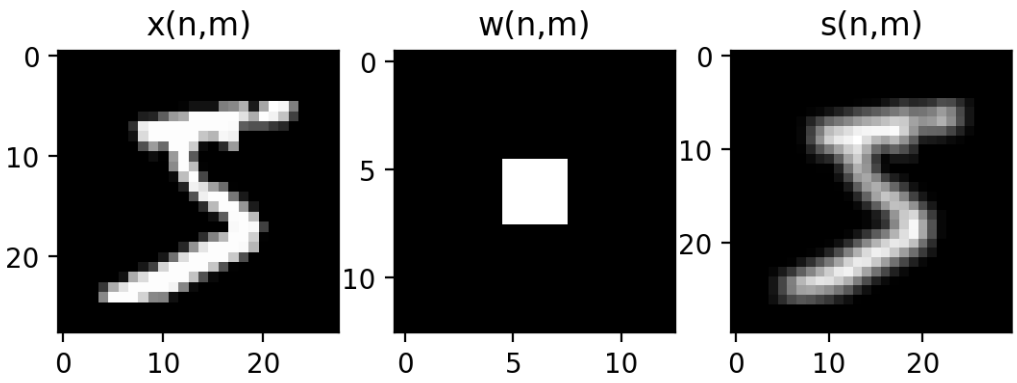
- Straightforward to extend to multiple dimensions
- 2D discrete convolution

$$s(n,m) = x * w = \sum_a \sum_b x(a,b)w(n-a, m-b)$$

[14]:
```
X = trainimg[0]
W = array([[1,1,1],[1,1,1], [1,1,1]])/9.

sfig = plt.figure()
plt.subplot(1,3,1)
plt.imshow(X, cmap='gray')
plt.title('x(n,m)')
plt.subplot(1,3,2)
plt.imshow(zeropad2(W, 5, 5, 5, 5), cmap='gray')
plt.title('w(n,m)')
plt.subplot(1,3,3)
plt.imshow(signal.convolve2d(X,W), cmap='gray')
plt.title('s(n,m)')
plt.close()
```

[15]:
```
sfig
```

[15]:



9 Two Interpretations of Convolution

- Interpretation 1:
  - w is a filter on the frequency spectrum of signal x.
    - Example filters: *low-pass*, *high-pass*, *band-pass*, *moving-average*
- Analysis is in the frequency domain:
  - **Time domain** $x(t) \iff$ **Frequency domain** $X(\omega)$
- Discrete-time Fourier transform (DTFT)
  - represent data as sum of complex exponentials basis functions with different frequencies.
    - $e^{-i\omega t} = \cos(\omega t) - i\sin(\omega t)$
    - Imaginary number: $i^2 = -1$
  - $X(\omega) = \sum_t x(t)e^{-i\omega t} \iff x(t) = \frac{1}{2\pi}\int X(\omega)e^{i\omega t}d\omega$
- **Key results:**
  - convolution in the time domain is equivalent to multiplication in the frequency domain:
    - $s(t) = x(t) * w(t) \iff S(\omega) = X(\omega)W(\omega)$
  - we can design and analyze filters in the frequency domain, and then obtain their time-domain representation.

[16]:
```
import numpy as np
from scipy.signal import butter, lfilter, freqz
import matplotlib.pyplot as plt
from scipy.fftpack import fft as fft

fs = 30.0       # sample rate, Hz

# Demonstrate the use of the filter.
# First make some data to be filtered.
T = 5.0         # seconds
n = int(T * fs) # total number of samples
t = np.linspace(0, T, n, endpoint=False)
# "Noisy" data.  We want to recover the 1.2 Hz signal from this.
```

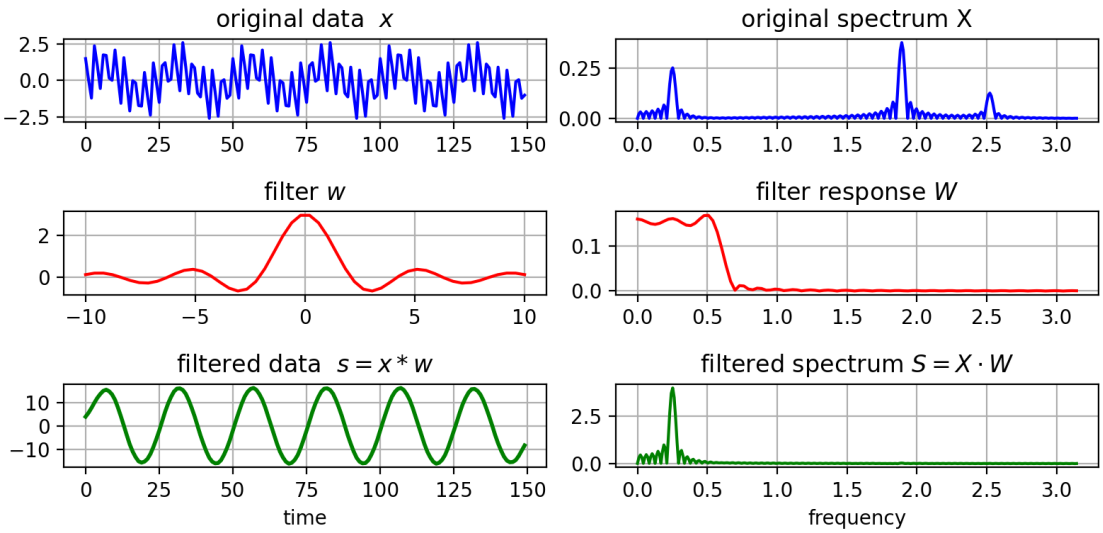
```
data = np.sin(1.2*2*np.pi*t) + 1.5*np.cos(9*2*np.pi*t) + 0.5*np.sin(12.0*2*np.
 ↪pi*t)


tt = linspace(-10,10)
w = sin(3.*tt/2) /  (tt/2);

y = convolve(data, w, mode='same')
ffig = plt.figure(figsize=(8,4))

plt.subplot(3, 2, 1)
plt.plot(range(0,n), data, 'b-', label='data')
plt.grid()
plt.title('original data  $x$')


plt.subplot(3, 2, 5)
plt.plot(range(0,n), y, 'g-', linewidth=2, label='filtered data')
plt.xlabel('time')
plt.title('filtered data  $s=x*w$')
plt.grid()



def calculateFFT(time,signal):
    signal = concatenate([signal, zeros((3*len(signal),))])
    N=len(signal)
    #df=1/((time[-1]-time[0]))
    #frequencies=[i*df for i in range(int(N/2.0))]
    frequencies = linspace(0,pi,int(N/2.))
    fftValues = [2.0/N*abs(i) for i in fft(signal,N)[0:int(N/2.0)] ]
    return frequencies,fftValues

plt.subplot(3, 2, 2)
originalfreqs,originalFFT=calculateFFT(t,data)
plt.plot(originalfreqs,originalFFT,"b",label="original")
plt.grid()
plt.title('original spectrum X')

plt.subplot(3, 2, 6)
filteredfreqs,filteredFFT=calculateFFT(t,y)
plt.plot(filteredfreqs,filteredFFT,"g",label="filtered")
plt.grid()
plt.title('filtered spectrum $S=X\cdot W$')
plt.xlabel('frequency')
```

```
# Plot the frequency response.
#w, h = freqz(b, a, worN=8000)
#plt.subplot(3, 2, 4)
##plt.plot(0.5*fs*w/np.pi, np.abs(h), 'b')
#plt.plot(cutoff, 0.5*np.sqrt(2), 'ko')
#plt.axvline(cutoff, color='k')
#plt.xlim(0, 0.5*fs)

plt.subplot(3,2,3)
plt.plot(tt,w, 'r')
plt.grid()
plt.title('filter $w$')
plt.subplot(3,2,4)
wfreqs,wFFT=calculateFFT(tt,w)
plt.plot(wfreqs,wFFT,"r",label="filtered")
plt.title("filter response $W$")
plt.grid()

#plt.subplots_adjust(hspace=0.45)
plt.tight_layout()
plt.close()
```

```
<>:56: SyntaxWarning: invalid escape sequence '\c'
<>:56: SyntaxWarning: invalid escape sequence '\c'
/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_74841/2482302383.py:5
6: SyntaxWarning: invalid escape sequence '\c'
  plt.title('filtered spectrum $S=X\cdot W$')
```

[17]:
```
ffig
```

[17]:



- Relationships between time and frequency spectrum
  - short-duration (high-frequency) signal $\Leftrightarrow$ large frequency spectrum
  - long-duration (low-frequency) signal $\Leftrightarrow$ small frequency spectrum
- For filters...
  - short (small) filter only captures short-range correlations (high frequencies).
  - long (large) filter can capture long-range correlations (low frequencies).

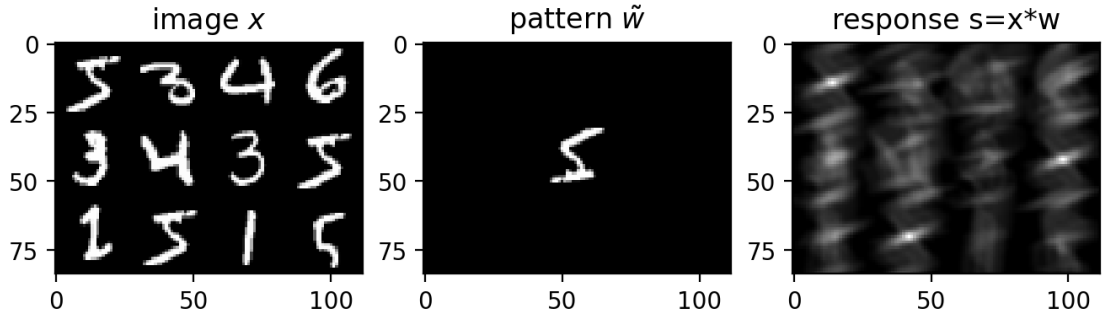
- Interpretation 2:
  - $\tilde{w}$ is a pattern (template); try to find this pattern.
  - the maximum correlation occurs when pattern $\tilde{w}$ matches the local x.
    - for a fixed energy $||x||^2 = 1$, $x = \frac{\tilde{w}}{||\tilde{w}||}$ has the maximum correlation with (response to) $\tilde{w}$.

[18]:
```
X = trainimg[0].astype('float64')
W = fliplr(flipud(X.astype('float64')))

XX = concatenate(
    [
        concatenate(trainimg[[0,3,6,9]], axis=1),
        concatenate(trainimg[[1,2,5,0]], axis=1),
        concatenate(trainimg[[7,0,4,10]], axis=1)
    ],
    axis=0
).astype('float64')
```

[19]:
```
pfig = plt.figure(figsize=(8,6))
plt.subplot(1,3,1)
plt.imshow(XX, cmap='gray')
plt.title('image $x$')
plt.subplot(1,3,2)
plt.imshow(zeropad2(W, 42, 42, 28, 28), cmap='gray')
plt.title('pattern $\\tilde{w}$')
plt.subplot(1,3,3)
plt.imshow(signal.convolve2d(XX,W,mode='same'), cmap='gray')
plt.title('response s=x*w')
plt.close()
```

[20]:
```
pfig
```

[20]:



10 Convolution as a layer

- output layer is the convolution of input with filter (kernel) $\mathbf{w}$.
  - $\mathbf{z} = \mathbf{x} * \mathbf{w}$.
- filter $\mathbf{w}$ acts locally on input $\mathbf{x}$.
  - $\mathbf{w}$ also called a **kernel**.
- Equivalent to a linear transformation (layer) where A has a particular form.
  - For example, if $\mathbf{w} = [w_1, w_2, w_3]$ and using "same" mode,

$$\begin{aligned}\mathbf{z} &= \mathbf{x} * \mathbf{w} \\ &= \mathbf{A}^T\mathbf{x} = \begin{bmatrix} w_2 & w_3 & 0 & 0 & 0 & 0 & \cdots \\ w_1 & w_2 & w_3 & 0 & 0 & 0 & \cdots \\ 0 & w_1 & w_2 & w_3 & 0 & 0 & \cdots \\ 0 & 0 & w_1 & w_2 & w_3 & 0 & \cdots \\ \cdots & & & & & & \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_N \end{bmatrix}\end{aligned}$$

  - Note: $\mathbf{A}$ has size $|\mathbf{x}||\mathbf{z}|$, but only $|\mathbf{w}|$ parameters.

11 Translation equivariance

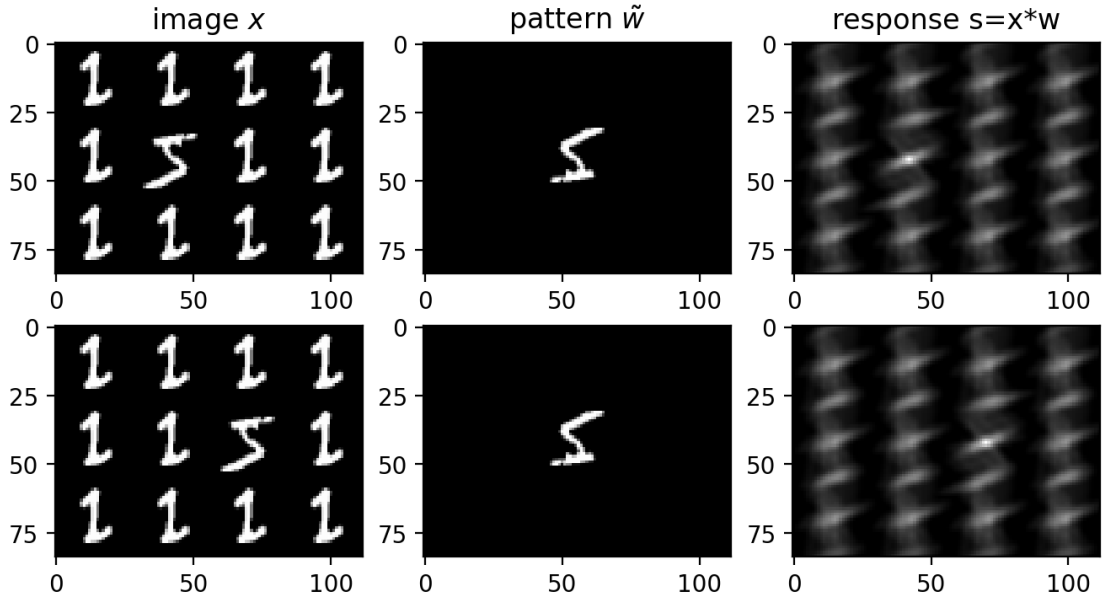
- Shifting x also shifts the response s.
- if $s = x * w$,
  - then $s(t - a) = x(t - a) * w(t)$
- We can find the pattern everywhere in x using the same filter.

[21]:
```
X = trainimg[0].astype('float64')
W = fliplr(flipud(X.astype('float64')))

XX = concatenate(
    [
        concatenate(trainimg[[7,7,7,7]], axis=1),
        concatenate(trainimg[[7,0,7,7]], axis=1),
        concatenate(trainimg[[7,7,7,7]], axis=1)
    ],
    axis=0
).astype('float64')


XX2 = concatenate(
    [
        concatenate(trainimg[[7,7,7,7]], axis=1),
        concatenate(trainimg[[7,7,0,7]], axis=1),
        concatenate(trainimg[[7,7,7,7]], axis=1)
    ],
```

```
    axis=0
).astype('float64')


cfig = plt.figure(figsize=(8,4))
plt.subplot(2,3,1)
plt.imshow(XX, cmap='gray')
plt.title('image $x$')
plt.subplot(2,3,2)
plt.imshow(zeropad2(W, 42, 42, 28, 28), cmap='gray')
plt.title('pattern $\\tilde{w}$')
plt.subplot(2,3,3)
plt.imshow(signal.convolve2d(XX,W,mode='same'), cmap='gray')
plt.title('response s=x*w')

plt.subplot(2,3,4)
plt.imshow(XX2, cmap='gray')
plt.subplot(2,3,5)
plt.imshow(zeropad2(W, 42, 42, 28, 28), cmap='gray')
plt.subplot(2,3,6)
plt.imshow(signal.convolve2d(XX2,W,mode='same'), cmap='gray')
plt.close()
```

[22]:
```
cfig
```

[22]:



12 Convolutional Neural Network (CNN)

- series of convolutional layers, sub-sampling layers, and MLP classifier.
  - convolutional and subsampling layers extract image features.
  - MLP uses extracted features for classification.

13 2D Convolution

- Use the spatial structure of the image
- 2D convolution filter
  - the weights **W** form a 2D filter template
  - filter response: $h = f(\sum_{x,y} W_{x,y} P_{x,y})$
    - **P** is an image patch with the same size as **W**.
- Convolution feature map
  - pass a sliding window over the image, and apply filter to get a *feature map*.
- Convolution modes
  - **"valid"** mode - only compute feature where convolution filter has valid image values.
    - size of feature map is reduced.
- Convolution modes
  - **"same"** mode - zero-pad the border of the image
    - feature map is the same size as the input image.
- Usually "same" is better since it looks at structures around border.
  - position information is implicitly encoded in the CNN features based on the zero-padding border.

14 2D Convolutional layer

- **Input:** HxW image with C channels
  - For example, in the first layer, C=3 for RGB channels.
  - defines a 3D volume: C x H x W (or H x W x C)
- **Features:** apply F convolution filters to get F feature maps.
  - Each feature map uses a 3D convolution filter (CxKxK) on the input
  - K is the spatial extent of the filter; total FCKK parameters
- **Activation:**
  - an activation function can be applied before output
- **Output:** a feature map with F channels
  - defines a 3D volume: F x H' x W'
  - H' and W' depend on various factors.

15 Combining Convolutional Layers

- Concatenate several convolutional layers.
  - From layer to layer
    - spatial resolution decreases
    - number of feature maps increases
  - Can extract high-level features in the final layers

- Feature map representation:

16 Spatial sub-sampling

- reduce the feature map size by subsampling feature maps between convolutional layers
  - *stride* for convolution filter - step size when moving the windows across the image.
- Spatial sub-sampling
  - *max-pooling layer* - use the maximum over a pooling window
    - gathers features together, summarizes features in local region.
    - introduces **translation invariance**
      - it doesn't matter where the maximal feature is located locally, it is passed to the next layer.
      - increases robustness to small changes in configuration of features

17 Receptive field size

- Stacking convolutional layers increases the effective size of the pattern filter
  - called **receptive field** - what pixels in the input affect a particular node.
  - larger receptive fields can see larger patterns.
  - Example: 2 convolutional layers
    - $|\mathbf{w_1}| = 5, |\mathbf{w_2}| = 3$, receptive field size $= |\mathbf{w_1}| + |\mathbf{w_2}| - 1 = 7$

18 Advantages of Convolution Layers

- The convolutional filters extract the same features throughout the image.
  - Useful because the object can appear in different locations of the image (global translation equivarience).
- Pooling makes it robust to changes in feature configuration (local translation invariance).
- The number of parameters is small compared to Dense (Fully-connected) layer
  - Example: input is C x H x W, and output is F x H x W
    - Number of MLP parameters: (CHW+1) x (FHW)
    - Number of CNN parameters: F x (CKK+1)

19 Fully-connected layers (MLP)

- after several convolutional layers, input the feature map into an MLP to get the final classification.
- also called "fully-connected" (FC) layers.

20 Example: Object classification CNN

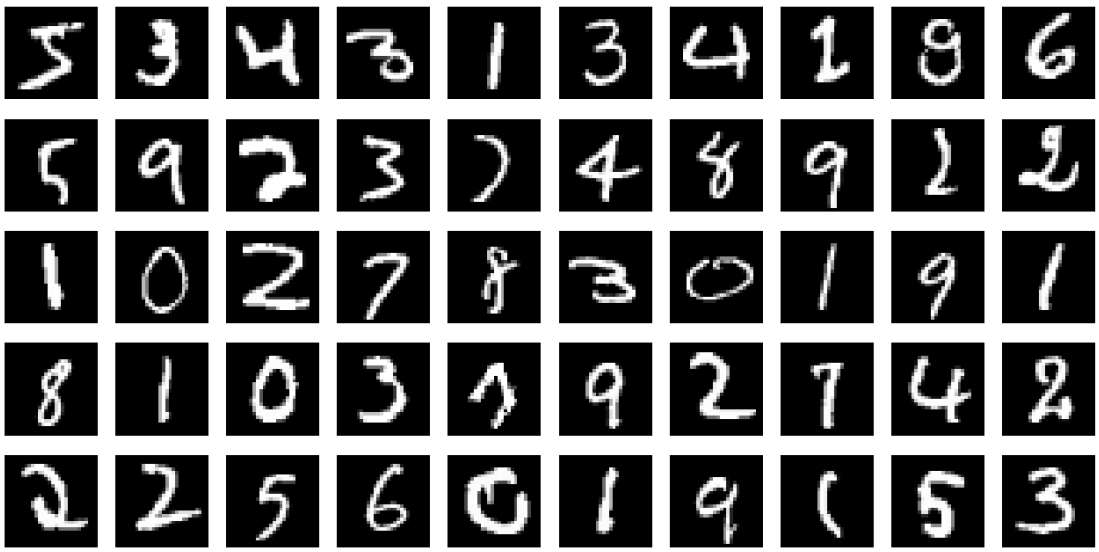
- Each layer shows its feature maps for the example image.
  - early layers extract *low-level* (visual) features
    - e.g., corners, edges
  - middle layers extract *mid-level* (part) features

        * e.g., object parts
    - later layers extract *high-level* (semantic) features.
        * e.g., object

- The number of feature channels increases with each layer
    - combining low-level parts together to get more higher-level parts
        * e.g., {edges, corners} -> {wheel, door, window}
    - trading off spatial resolution for semantic specificity
        * e.g., 512x512x3 RGB image -> 8x8x512 semantic features
- the spatial resolution decreases with each layer
    - increase the window size (receptive field) on the object
    - high-level semantic correspond to large regions.

21 CNN on MNIST

[23]:
```
# Example images
plt.figure(figsize=(8,4))
show_imgs(trainimg[0:50])
```



22 4D tensor format

- There are two common formats for the 4-D tensor:
    - "NCHW" - batch, channel, height, width - called `'channels_first'`
    - "NHWC" - batch, height, width, channel - called `'channels_last'`
- NCHW is required for PyTorch while NHWC is required for CPU version of Tensorflow 2.

23 Example on MNIST

- Pre-processing
  - scale to [0,1]
  - 4-D tensor: (sample, height, width, channel)
    - channel = 1 (grayscale)
  - create training/validation sets

[24]:
```
# Convert to PyTorch tensors and scale to 0-1
trainI = torch.from_numpy(trainimg.reshape((6000, 1, 28, 28)) / 255.0).float()
testI = torch.from_numpy(testimg.reshape((10000, 1, 28, 28)) / 255.0).float()

print(trainI.shape)
print(testI.shape)
```

```
torch.Size([6000, 1, 28, 28])
torch.Size([10000, 1, 28, 28])
```

- Generate fixed training and validation sets

[25]:
```
# Generate fixed validation set
from sklearn.model_selection import train_test_split

vtrainI_np, validI_np, vtrainY_np, validY_np = train_test_split(
    trainI.numpy(), trainY,
    train_size=0.9, test_size=0.1, random_state=4487
)

vtrainI = torch.from_numpy(vtrainI_np).float()
validI = torch.from_numpy(validI_np).float()
vtrainY = torch.from_numpy(vtrainY_np).long()
validY = torch.from_numpy(validY_np).long()

print(f"Training set: {vtrainI.shape}, {vtrainY.shape}")
print(f"Validation set: {validI.shape}, {validY.shape}")

cid, cfreq = vtrainY.unique(return_counts=True)

print(f"Training set Distribution:", cfreq.numpy().tolist())
```

```
Training set: torch.Size([5400, 1, 28, 28]), torch.Size([5400])
Validation set: torch.Size([600, 1, 28, 28]), torch.Size([600])
Training set Distribution: [491, 599, 554, 556, 513, 518, 567, 584, 496, 522]
```

24 Shallow CNN Architecture

- 1 Convolution layer
  - 5x5x1 kernel, 10 features, *stride* = 2 (step-size between sliding windows)
  - No pooling here since the image input is small (28x28)

  - Input: 28x28x1 (grayscale image) -> Output: 14x14x10
- 1 fully-connected layer (MLP), 50 nodes
  - Input: 14x14x10=1960 -> Output: 50
- Classification output node

[26]:
```
# Build the network
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5, stride=2, padding=2)  #␣
 ↪input channels=1, output channels=10
        self.flatten = nn.Flatten()

        # Calculate the size after conv layers for the dense layers
        # Input: 28x28, after conv: (28/2)=14x14, flattened: 10*14*14 = 1960
        self.fc1 = nn.Linear(10 * 14 * 14, 50)  # units=50
        self.fc2 = nn.Linear(50, 10)  # units=10

        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        # Note: We'll use CrossEntropyLoss which includes softmax, so we don't␣
 ↪apply it here
        return x
```

[27]:
```
# Early stopping implementation
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.0001, verbose=1):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_accuracy = 0.0
        self.early_stop = False
        self.verbose = verbose

    def __call__(self, current_accuracy):
        if current_accuracy - self.best_accuracy > self.min_delta:
            self.best_accuracy = current_accuracy
            self.counter = 0
        else:
            self.counter += 1
            if self.verbose:
```

```
                print(f'EarlyStopping counter: {self.counter} out of {self.
↪patience}')
            if self.counter >= self.patience:
                self.early_stop = True
        return self.early_stop
```

[28]:
```
# Model Parameter Summarization
def summarize_layer_parameters(model):
    """
    Summarizes the number of parameters in each layer of a PyTorch model.
    """
    trainable_table = PrettyTable(["Module Name", "Parameters"])
    untrainable_table = PrettyTable(["Module Name", "Parameters"])
    total_params = {'train':0,'untrain':0}
    for name, parameter in model.named_parameters():
        if parameter.requires_grad:  # Only count trainable parameters
            params = parameter.numel()
            trainable_table.add_row([name, params])
            total_params['train'] += params
        else:  # Only count untrainable parameters
            params = parameter.numel()
            untrainable_table.add_row([name, params])
            total_params['untrain'] += params

    print(trainable_table)
    print(f"\nTotal Trainable Parameters: {total_params['train']}")
    if total_params['untrain'] > 0:
        print(untrainable_table)
    print(f"\nTotal Un-Trainable Parameters: {total_params['untrain']}")
```

[29]:
```
# Model Evaluation
def evaluate_model(model, eval_loader, criterion=None):
    model.eval()
    eval_correct = 0
    eval_total = 0
    eval_loss = 0.0

    with torch.no_grad():
        for data, target in eval_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            _, predicted = torch.max(output.data, 1)
            eval_total += target.size(0)
            eval_correct += (predicted == target).sum().item()

            if criterion is not None:
                loss = criterion(output, target)
```

```
                eval_loss += loss.item()

    eval_accuracy = 100 * eval_correct / eval_total
    avg_eval_loss = eval_loss / len(eval_loader)

    return eval_accuracy if criterion is None else (eval_accuracy,␣
 ↪avg_eval_loss)


# Model Training
def train_model(model, train_loader, valid_loader, criterion, optimizer,␣
 ↪epochs=100, patience=5):
    history = {
        'train_loss': [], 'val_loss': [],
        'train_accuracy': [], 'val_accuracy': []
    }
    early_stopping = EarlyStopping()

    # best_val_loss = float('inf')
    # patience_counter = 0

    for epoch in range(epochs):
        # Training phase
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0

        for batch_idx, (data, target) in enumerate(train_loader):
            data, target = data.to(device), target.to(device)

            optimizer.zero_grad()
            output = model(data)

            # loss = criterion(output, torch.nn.functional.one_hot(target,␣
 ↪num_classes=10).float()) also works
            loss = criterion(output, target)

            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            _, predicted = torch.max(output.data, 1)
            train_total += target.size(0)
            train_correct += (predicted == target).sum().item()

        train_accuracy = 100 * train_correct / train_total
```

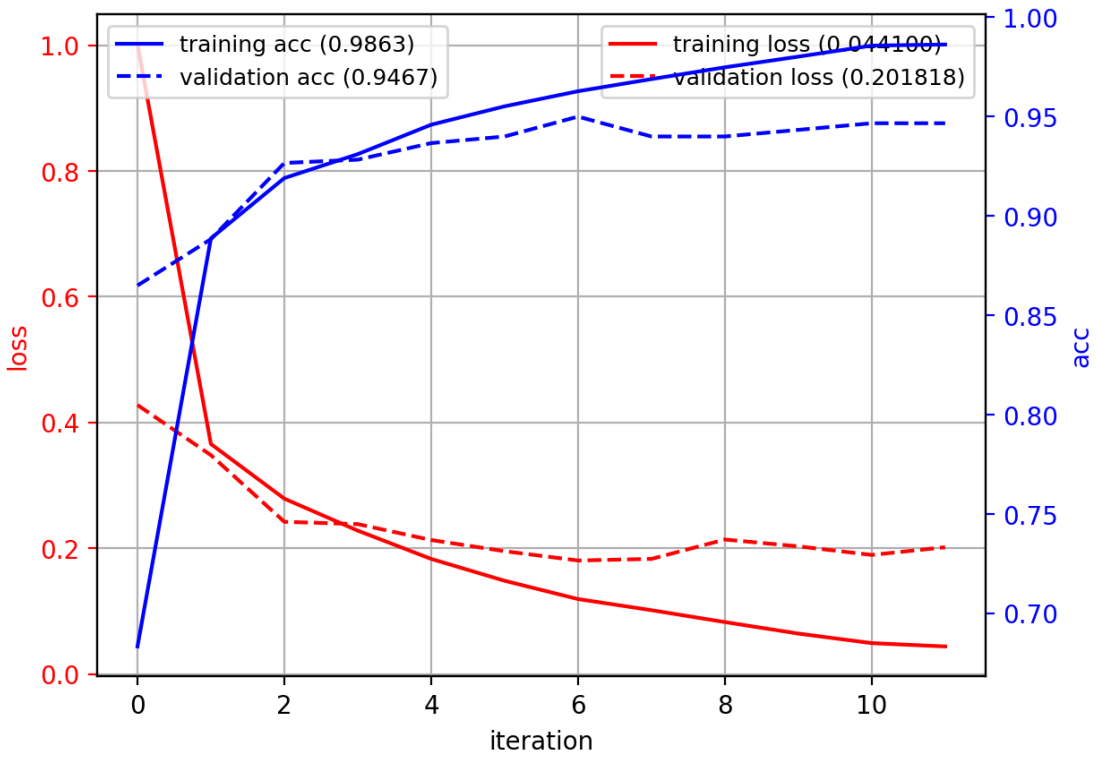
```
        avg_train_loss = train_loss / len(train_loader)

        # Validation phase
        val_accuracy, avg_val_loss =␣
↪evaluate_model(model,valid_loader,criterion)

        # Store history
        history['train_loss'].append(avg_train_loss)
        history['val_loss'].append(avg_val_loss)
        history['train_accuracy'].append(train_accuracy / 100)
        history['val_accuracy'].append(val_accuracy / 100)

        train_msg = f'  Train Loss: {avg_train_loss:.6f}, Train Acc:␣
↪{train_accuracy:.2f}%;'
        valid_msg = f'  Val Loss: {avg_val_loss:.6f}, Val Acc: {val_accuracy:.
↪2f}%'

        print(f'Epoch {epoch+1}/{epochs}:', train_msg, valid_msg)

        # Check early stopping
        early_stopping(val_accuracy)
        if early_stopping.early_stop:
            print("Early stopping triggered")
            break
        else:
            torch.save(model.state_dict(), 'best_model.pth')

    # Load best model
    model.load_state_dict(torch.load('best_model.pth'))
    return history, model
```

[30]:
```
# Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True

# Create model instance
model = SimpleCNN()

# Print model architecture
print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")


# Prepare data - ensure correct shape for PyTorch (channel first)
# Assuming vtrainI, vtrainY, validI, validY are already defined as in previous␣
 ↪cells

# Convert from (batch, height, width, channels) to (batch, channels, height,␣
 ↪width)
if len(vtrainI.shape) == 4 and vtrainI.shape[-1] == 1:  # If shape is (N, 28,␣
 ↪28, 1)
    vtrainI = vtrainI.transpose(0, 3, 1, 2)
    validI = validI.transpose(0, 3, 1, 2)

# Create DataLoaders
batch_size = 50
train_dataset = TensorDataset(vtrainI, vtrainY)
valid_dataset = TensorDataset(validI, validY)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Compile the network (equivalent in PyTorch)
criterion = nn.CrossEntropyLoss()  # This includes softmax + cross entropy
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,␣
 ↪optimizer, epochs=100)

# Plot the training history
plot_history(history)
```

```
SimpleCNN(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=1960, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
)
+--------------+------------+
| Module Name  | Parameters |
+--------------+------------+
| conv1.weight |    250     |
```

```
|  conv1.bias  |     10     |
|  fc1.weight  |   98000    |
|   fc1.bias   |     50     |
|  fc2.weight  |    500     |
|   fc2.bias   |     10     |
+--------------+------------+

Total Trainable Parameters: 98820

Total Un-Trainable Parameters: 0
Using device: cpu
Epoch 1/100:   Train Loss: 1.000959, Train Acc: 68.33%;   Val Loss: 0.427933,
Val Acc: 86.50%
Epoch 2/100:   Train Loss: 0.365901, Train Acc: 88.87%;   Val Loss: 0.348270,
Val Acc: 88.83%
Epoch 3/100:   Train Loss: 0.279040, Train Acc: 91.91%;   Val Loss: 0.242267,
Val Acc: 92.67%
Epoch 4/100:   Train Loss: 0.228215, Train Acc: 93.11%;   Val Loss: 0.238575,
Val Acc: 92.83%
Epoch 5/100:   Train Loss: 0.183257, Train Acc: 94.59%;   Val Loss: 0.213258,
Val Acc: 93.67%
Epoch 6/100:   Train Loss: 0.148498, Train Acc: 95.52%;   Val Loss: 0.195522,
Val Acc: 94.00%
Epoch 7/100:   Train Loss: 0.119439, Train Acc: 96.28%;   Val Loss: 0.180730,
Val Acc: 95.00%
Epoch 8/100:   Train Loss: 0.101738, Train Acc: 96.89%;   Val Loss: 0.183262,
Val Acc: 94.00%
EarlyStopping counter: 1 out of 5
Epoch 9/100:   Train Loss: 0.082846, Train Acc: 97.48%;   Val Loss: 0.214063,
Val Acc: 94.00%
EarlyStopping counter: 2 out of 5
Epoch 10/100:   Train Loss: 0.064539, Train Acc: 98.02%;   Val Loss: 0.203230,
Val Acc: 94.33%
EarlyStopping counter: 3 out of 5
Epoch 11/100:   Train Loss: 0.049383, Train Acc: 98.57%;   Val Loss: 0.189592,
Val Acc: 94.67%
EarlyStopping counter: 4 out of 5
Epoch 12/100:   Train Loss: 0.044100, Train Acc: 98.63%;   Val Loss: 0.201818,
Val Acc: 94.67%
EarlyStopping counter: 5 out of 5
Early stopping triggered
```



[31]:
```
from torch.utils.tensorboard import SummaryWriter

# Create a writer
writer = SummaryWriter()

# Add model graph to tensorboard
dummy_input = torch.randn(1, 1, 28, 28).to(device)
writer.add_graph(model, dummy_input)
writer.close()

print("Model graph saved to TensorBoard. Run: tensorboard --logdir=runs")
```

```
Model graph saved to TensorBoard. Run: tensorboard --logdir=runs
```

[32]:
```
test_dataset = TensorDataset(testI, torch.from_numpy(testY).long())
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
acc = evaluate_model(model, test_loader)

print("test accuracy:", acc)
```

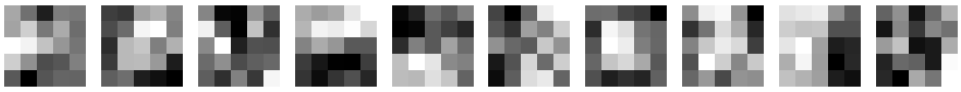
```
test accuracy: 94.22
```

- Visualize the convolutional filters

- filters are looking for local stroke features
  - corners, edges, lines

[33]:
```
W = model.conv1.weight.data.cpu()
print(W.shape)
flist = [squeeze(W[c]) for c in range(10)]
show_imgs(flist)
```

```
torch.Size([10, 1, 5, 5])
```



25 Deep CNN Architecture

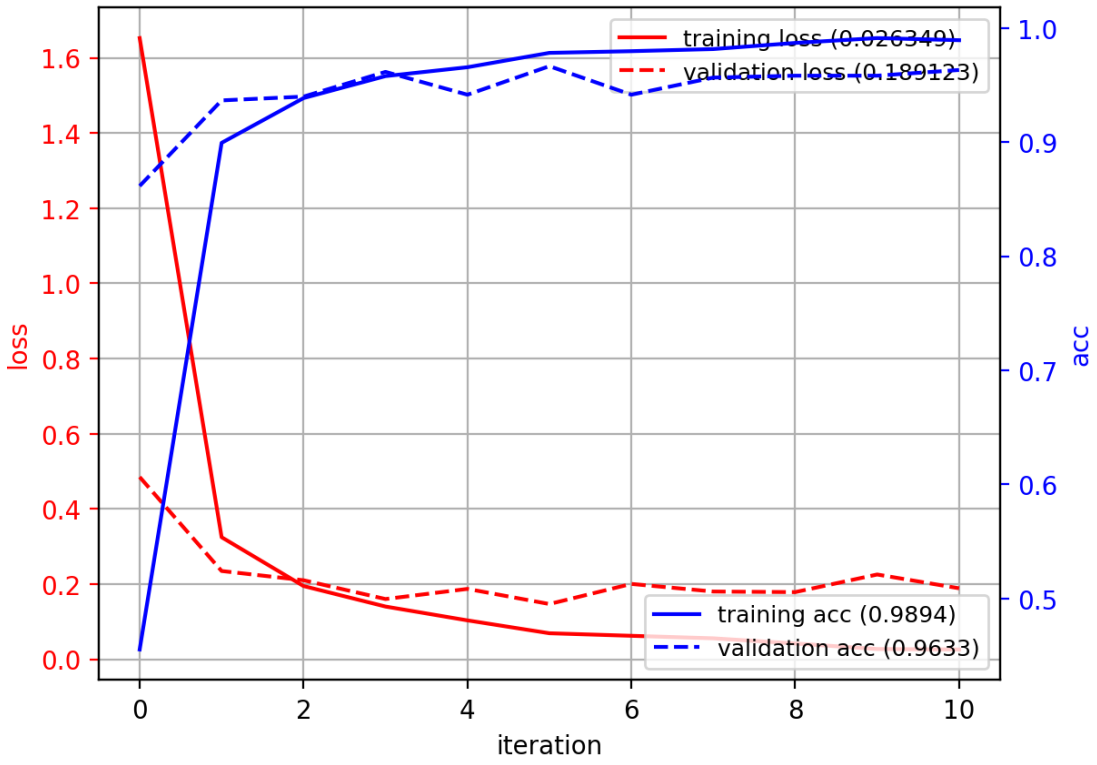
- 3 Convolutional layers
  - 5x5x1 kernel, stride 2, 10 features (output 14x14x10)
  - 5x5x10 kernel, stride 2, 40 features (output 7x7x40)
  - 5x5x40 kernel, stride 1, 80 features (output 7x7x80)
  - set stride as 1 to avoid reducing the feature map too much)
- 1 fully-connected layer (7x7x80=3920 -> 50)
- Classification output node

[34]:
```
class DeepCNN(nn.Module):
    def __init__(self, weight_decay=0.0001):
        super(DeepCNN, self).__init__()
        self.weight_decay = weight_decay

        self.conv1 = nn.Conv2d(1, 10, kernel_size=5, stride=2, padding=2)
        self.conv2 = nn.Conv2d(10, 40, kernel_size=5, stride=2, padding=2)
        self.conv3 = nn.Conv2d(40, 80, kernel_size=5, stride=1, padding=2)
        self.fc1 = nn.Linear(80 * 7 * 7, 50)  # Adjusted based on input size␣
 ↪and strides
        self.fc2 = nn.Linear(50, 10)

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.relu(self.conv3(x))
        x = x.view(x.size(0), -1)  # Flatten
```

```
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = DeepCNN()

# Print model architecture
print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,␣
 ↪optimizer, epochs=100)

# Plot the training history
plot_history(history)
```

```
DeepCNN(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.5, inplace=False)
  (softmax): Softmax(dim=1)
)
Using device: cpu
Epoch 1/100:   Train Loss: 1.652544, Train Acc: 45.54%;   Val Loss: 0.485118,
Val Acc: 86.17%
Epoch 2/100:   Train Loss: 0.324927, Train Acc: 89.94%;   Val Loss: 0.235094,
Val Acc: 93.67%
Epoch 3/100:   Train Loss: 0.195066, Train Acc: 93.87%;   Val Loss: 0.210481,
Val Acc: 94.00%
Epoch 4/100:   Train Loss: 0.140573, Train Acc: 95.80%;   Val Loss: 0.160527,
Val Acc: 96.17%
Epoch 5/100:   Train Loss: 0.103770, Train Acc: 96.57%;   Val Loss: 0.187480,
Val Acc: 94.17%
EarlyStopping counter: 1 out of 5
Epoch 6/100:   Train Loss: 0.069524, Train Acc: 97.83%;   Val Loss: 0.147277,
Val Acc: 96.67%
```

```
Epoch 7/100:   Train Loss: 0.062744, Train Acc: 97.98%;   Val Loss: 0.200751,
Val Acc: 94.17%
EarlyStopping counter: 1 out of 5
Epoch 8/100:   Train Loss: 0.056061, Train Acc: 98.17%;   Val Loss: 0.180467,
Val Acc: 95.67%
EarlyStopping counter: 2 out of 5
Epoch 9/100:   Train Loss: 0.042313, Train Acc: 98.70%;   Val Loss: 0.178956,
Val Acc: 95.83%
EarlyStopping counter: 3 out of 5
Epoch 10/100:   Train Loss: 0.027585, Train Acc: 99.13%;   Val Loss: 0.225759,
Val Acc: 95.83%
EarlyStopping counter: 4 out of 5
Epoch 11/100:   Train Loss: 0.026349, Train Acc: 98.94%;   Val Loss: 0.189123,
Val Acc: 96.33%
EarlyStopping counter: 5 out of 5
Early stopping triggered
```



[35]:
```
acc = evaluate_model(model, test_loader)
print("test accuracy: ", acc)
```

```
test accuracy:  95.97
```

26 Summary

- Convolution operation
  - looks at the local structure of the signal (1D, 2D, 3D, etc).
  - two intepretations: filtering, pattern matching
- Convolutional neural network (CNN)
  - convolutional layer - use convolution instead of dense connections
  - learns to extract image features, and learns classifier.